PUC | Teste de Software

Detecção de Bad Smells e Refatoração Segura

Trabalho prático 03 | Serviço de geração de relatórios em JavaScript

Gabriel Chagas Lage

Matrícula: 803292

Repositório da entrega:
github.com/gabrielchagas13/bad-smells-js-refactoring

Data: 2 de novembro de 2026

Base analisada: CleitonSilvaT/bad-smells-js-refactoring
Commit: 39d39f9ad52c6c508239cfe0b968187ad086e654

1. Análise manual e detecção automática

Método longo e múltiplas responsabilidades. generateReport mistura cabeçalho, autorização, prioridade, formatação e totalização. Uma mudança de apresentação exige percorrer regras de negócio; testar uma regra isolada fica difícil e os cenários se multiplicam.

Condicionais aninhadas. O laço combina papel do usuário, valor do item e formato. Esse encadeamento aumenta o esforço para entender caminhos e identificar combinações não exercitadas pelos testes. O SonarJS mediu complexidade cognitiva 27, acima do limite 15.

Duplicação de lógica. A linha CSV e a soma do total aparecem nos caminhos ADMIN e USER; a estrutura HTML também se repete. Alterar só um caminho pode gerar inconsistência entre os relatórios e exigir casos de teste redundantes.

Observação adicional. Os valores 500 e 1000 são números mágicos: representam limites de visibilidade e prioridade. Constantes com nomes tornam a intenção explícita.

Configuração: ESLint 8 e SonarJS 0.21, com eslint:recommended e plugin:sonarjs/recommended. Foram mantidas as regras adicionais da base; a complexidade foi ajustada de 5 para 15, conforme o enunciado. O original continua com dois erros.

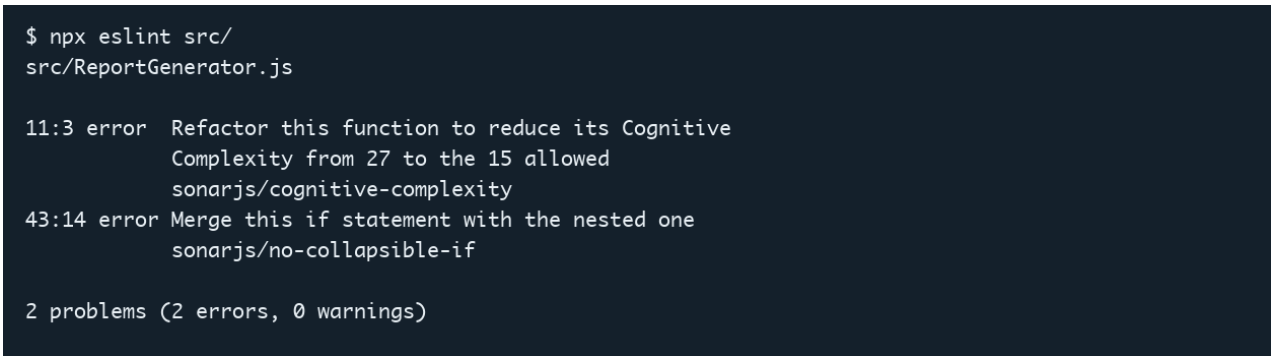


Figura 1. Reprodução visual da saída real capturada em evidencias/eslint-antes.txt; não é captura de tela. O SonarJS quantificou o aninhamento percebido manualmente e indicou a possibilidade de juntar condicionais. A duplicação e os números mágicos foram achados manuais, sem alerta específico nesta execução.

2. Processo de refatoração

O ponto mais crítico foi a combinação de autorização e formatação no mesmo método. Foram aplicadas Extract Method e Decompose Conditional: canViewItem decide visibilidade; markPriority concentra o efeito colateral; selectFormatter escolhe um objeto de estratégia para CSV ou HTML. A formatação deixa de ser repetida em cada papel.

Antes - trecho do ramo ADMIN em generateReport:

```
if (user.role === 'ADMIN') {
  if (item.value > 1000) {
    item.priority = true;
  }
  if (reportType === 'CSV') {
    report += `${item.id},${item.name},${item.value},${user.name}\n`;
    total += item.value;
  } else if (reportType === 'HTML') {
    // Formata HTML e soma novamente.
  }
} // O ramo USER repete formatação e soma.
```

Depois - fluxo compartilhado da versão refatorada:

```
for (const item of items) {
  markPriority(user, item);
  if (!formatter || !canViewItem(user, item)) continue;
  report += formatter.row(item, user);
  total += item.value;
}
```

Os limites receberam os nomes STANDARD_USER_VALUE_LIMIT (500) e ADMIN_PRIORITY_THRESHOLD (1000). O construtor e a mutação de item.priority foram preservados, inclusive no caso de formato desconhecido, que continua retornando texto vazio.

Correção adicional, separada da refatoração: o original emite <tr > para ADMIN sem prioridade; seu próprio teste exige <tr>. Na versão nova, o espaço faz parte do atributo opcional:

```
const style = user.role === 'ADMIN' && item.priority
  ? ' style="font-weight:bold;"' : '';
return `<tr${style}><td>${item.id}</td><td>${item.name}</td>` +
  `<td>${item.value}</td></tr>\n`;
```

Essa mudança textual corrige a falha existente; portanto não se afirma equivalência byte a byte com o original. Nenhum arquivo original de produção ou de teste foi alterado.

3. Validação, limites e conclusão

A instalação foi realizada com npm install. A primeira execução de npm test não carregava os imports ES Modules. O script passou a chamar o Jest com node --experimental-vm-modules, mantendo os testes intactos. Depois desse ajuste, a linha de base mostrou 4 testes aprovados e 1 falha preexistente no HTML de ADMIN.

Resultados efetivamente observados:

```
Original:                          4/5 testes aprovados
Refatorado:                        5/5 testes aprovados
Suíte completa:                   9/10 testes aprovados
ESLint original:                   2 erros
ESLint refatorado:                 0 erros
Comparação complementar:         162 cenários aprovados
```

A cópia do teste difere somente no import para ReportGenerator.refactored.js. O original continua falhando pelo espaço em <tr >; corrigir esse arquivo violaria a orientação de preservá-lo. Assim, a exigência de 100% da suíte completa não é atendida pela base fornecida. O relatório não apresenta essa falha como regressão nem como sucesso.

A comparação complementar percorreu três papéis (ADMIN, USER e OTHER), três formatos (CSV, HTML e INVALID), seis valores (0, 499, 500, 501, 1000 e 1001) e três estados de prioridade. Verificou igualdade de saída após normalizar somente <tr > e igualdade das mutações nos itens. Ela é reproduzível com node scripts/verificar-equivalencia.js.

As evidências dos testes e do lint estão em evidencias/. Os comandos npm run test:refactored e npm run lint:refactored validam a implementação nova. npm test -- --runInBand mantém visível a falha preexistente; npm run lint também mantém os erros do original.

Conclusão. Testes como rede de segurança precisam ser executados antes da mudança: foi essa etapa que revelou que a base não estava totalmente verde. A refatoração separou decisões e apresentação, reduziu o aninhamento e centralizou a lógica repetida. Os cinco testes fornecidos e a comparação adicional sustentam a mudança nos cenários cobertos, sem provar correção para todas as entradas possíveis. O erro do original deve ser discutido com o professor para conciliar sua preservação com a exigência de aprovação total.

Referências: enunciado “Detecção de Bad Smells e Refatoração Segura”; repositório-base CleitonSilvaT/bad-smells-js-refactoring; Fowler, Martin. Refatoração: Aperfeiçoando o Design de Códigos Existentes. 2. ed., Novatec, 2019 (referência indicada no enunciado).